Agent Spark — AI Agent Marketplace Technical PRD

> **Version:** 2.0.0 **Date:** February 27, 2026 **Status:** Build-Ready **Scope:** Complete technical specification for Claude Code execution

---

Table of Contents

---

1. Product Overview

Agent Spark is an open AI agent marketplace where users can browse, rent, and orchestrate AI agents that work together to automate business workflows. Anyone can build and publish agents. Users can either browse the marketplace to find agents manually, or describe what they need in plain English and let the platform assemble the right team of agents automatically.

Core User Experience

Entry Point 1 — Browse: User lands on the marketplace, sees agent cards organized by category, filters by use case/rating/price, clicks into an agent detail page, tries a sandbox demo, and rents it with one click.

Entry Point 2 — Describe: User types into a command bar: "I need help automating my Shopify returns and customer emails." The AI Workflow Assembler analyzes the request, searches the marketplace for matching agents, and presents a curated recommendation: "Here are the 3 agents I'd recommend for this workflow." User still sees the agents, still chooses, but the AI did the filtering.

After renting: User connects their business tools (Shopify, Gmail, Slack, etc.) via one-click OAuth. Agents use those connections to do real work. When agents need help from other agents, they communicate via the A2A protocol — the user sees this as a seamless team working together.

Core Concepts

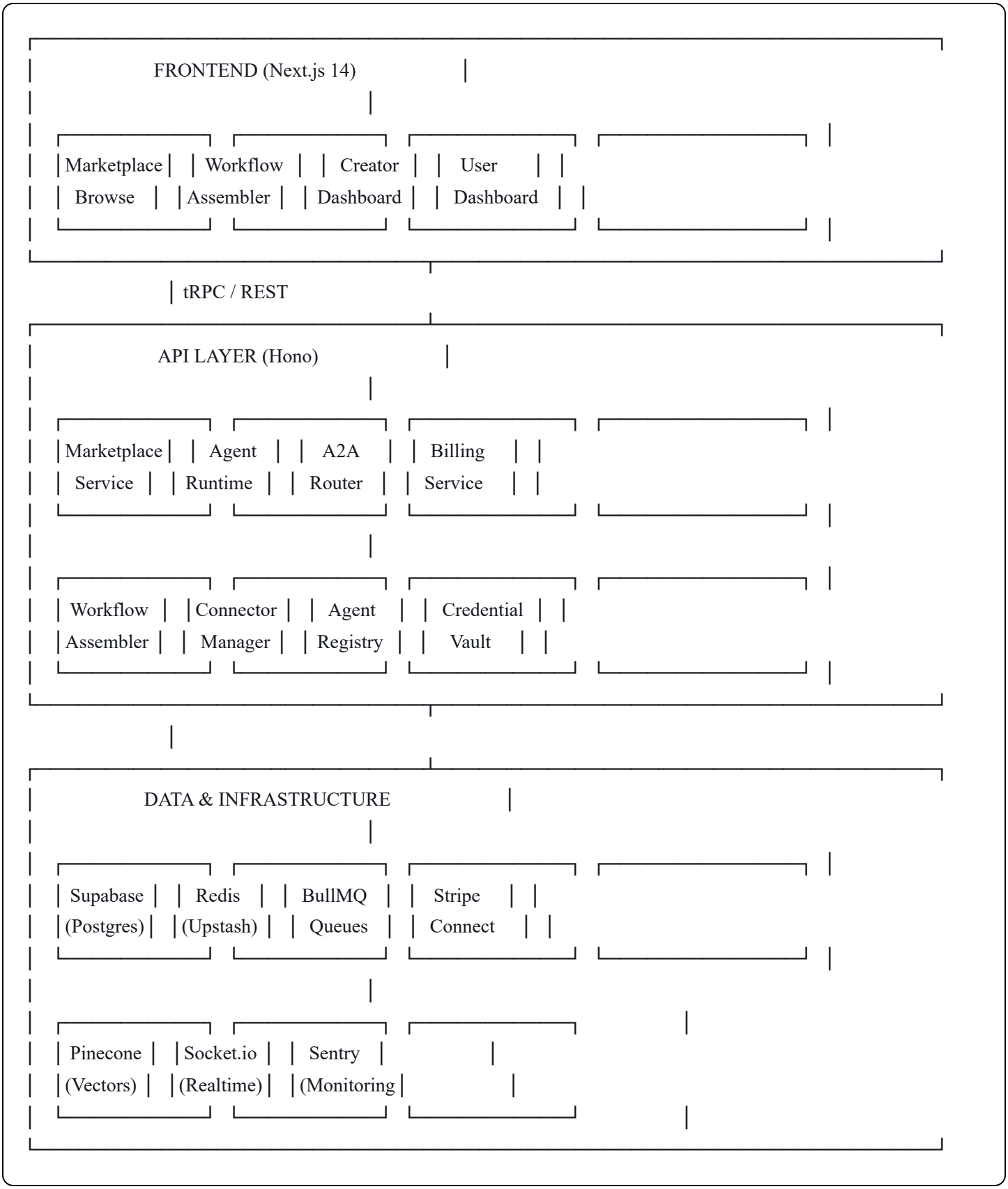
- **Agent** — An AI entity published to the marketplace with defined capabilities, pricing, and an A2A Agent Card. Built by any creator using the platform's agent builder.
- **Agent Card** — A2A protocol-compliant JSON manifest describing what an agent can do, what inputs it accepts, what it returns, and how to reach it. This is how agents discover and communicate with each other.
- **Connector** — A standardized adapter that wraps a third-party API (Gmail, Shopify, Slack, etc.) behind a typed interface. Users connect these via one-click OAuth. Agents use them to take real actions.
- **Rental** — When a user subscribes to use an agent. Creates a billing relationship and grants the agent access to the user's connected tools.
- **A2A Call** — When one agent delegates a subtask to another agent on the platform. Each call is a metered micro-transaction that earns revenue for the target agent's creator.
- **Workflow** — A set of agents working together to accomplish a user's goal. Can be manually assembled by the user or auto-assembled by the AI Workflow Assembler.
- **Creator** — Anyone who builds and publishes an agent to the marketplace. Earns revenue from rentals and A2A calls.

What This Is NOT

- Not a single-tenant agent tool (this is a multi-tenant marketplace)
- Not a chatbot builder (agents take real actions through connected tools)
- Not an API platform (end users are non-technical business owners)
- Not Web3 (V1 uses Stripe; blockchain payment layer planned for V2)

---

2. System Architecture



Key Architecture Decisions

- **Multi-tenant from day one.** Every table has org-level or user-level RLS. A creator's agent code never sees another user's data. Agent execution is scoped to the renter's connections and permissions only.

- **A2A as a connector.** Agent-to-agent communication is implemented as a special connector in the existing connector interface pattern. The agent runtime doesn't need to know if it's calling a Shopify API or another agent — it's all just tool calls.
- **Marketplace-first schema.** The database is designed around listings, rentals, and earnings — not around a single user's agents. Every agent has a public listing, a creator, and rental relationships.
- **Stateless agent runtime.** Agents don't hold state between calls. All context is passed in via the task payload. This means any instance of the runtime can execute any agent — critical for scaling.

---

3. Monorepo Structure

```
agentspark/
├── apps/
│   ├── web/                    # Next.js 14 (App Router) — Frontend
│   │   ├── app/
│   │   │   ├── (marketplace)/      # Public marketplace (no auth required)
│   │   │   │   ├── page.tsx        # Homepage = marketplace grid
│   │   │   │   ├── category/[slug]/
│   │   │   │   │   └── page.tsx    # Category filtered view
│   │   │   │   └── agent/[slug]/
│   │   │   │       └── page.tsx    # Agent detail + sandbox + rent CTA
│   │   │   ├── (dashboard)/        # Authenticated user area
│   │   │   │   ├── dashboard/
│   │   │   │   │   └── page.tsx    # My rented agents + usage meter
│   │   │   │   ├── connections/
│   │   │   │   │   └── page.tsx    # Connected tools (OAuth management)
│   │   │   │   ├── workflows/
│   │   │   │   │   └── page.tsx    # Active workflows list
│   │   │   │   ├── workflows/[id]/
│   │   │   │   │   └── page.tsx    # Workflow detail + network visualizer
│   │   │   │   └── settings/
│   │   │   │       └── page.tsx    # Account, billing, plan management
│   │   │   ├── (creator)/          # Creator Studio (separate sidebar mode)
│   │   │   │   ├── creator/
│   │   │   │   │   └── page.tsx    # Creator home — overview + earnings
│   │   │   │   ├── creator/publish/
│   │   │   │   │   └── page.tsx    # Smart publish flow (3-step AI listing)
│   │   │   │   ├── creator/listings/
│   │   │   │   │   └── page.tsx    # Manage published agents
│   │   │   │   ├── creator/earnings/
│   │   │   │   │   └── page.tsx    # Revenue charts + payout settings
```

```
│   │   │   │   └── creator/analytics/
│   │   │   │       └── page.tsx      # Per-agent usage stats
│   │   │   ├── api/                  # Next.js API routes (auth callbacks, webhooks)
│   │   │   └── layout.tsx            # Root layout with sidebar
│   │   ├── components/
│   │   │   ├── ui/                   # shadcn/ui components
│   │   │   ├── marketplace/          # AgentCard, AgentGrid, CategoryNav, SearchBar
│   │   │   ├── dashboard/            # WorkflowVisualizer, ConnectionCard, AgentTeam
│   │   │   ├── creator/              # AgentBuilder, EarningsChart, ListingEditor
│   │   │   └── shared/               # CommandBar, Layout, Navigation
│   │   ├── lib/
│   │   │   ├── supabase/             # Client, server, middleware helpers
│   │   │   ├── api/                  # tRPC or REST client
│   │   │   └── utils/
│   │   ├── styles/
│   │   │   └── globals.css           # Design system tokens
│   │   └── public/
│   │
│   └── api/                          # Hono API server — Backend
│       ├── src/
│       │   ├── routes/
│       │   │   ├── marketplace.ts    # Browse, search, filter, detail
│       │   │   ├── agents.ts         # CRUD, publish, sandbox
│       │   │   ├── rentals.ts        # Subscribe, cancel, manage
│       │   │   ├── connections.ts    # OAuth flows, token management
│       │   │   ├── workflows.ts      # Create, execute, monitor
│       │   │   ├── a2a.ts            # A2A protocol endpoints
│       │   │   ├── billing.ts        # Stripe webhooks, earnings
│       │   │   └── auth.ts           # Session validation
│       │   ├── services/
│       │   │   ├── agent-runtime.ts  # Core agentic loop
│       │   │   ├── agent-registry.ts # A2A agent discovery
│       │   │   ├── workflow-assembler.ts # AI-powered agent selection
│       │   │   ├── connector-manager.ts  # Load & manage connectors
│       │   │   ├── credential-vault.ts   # Encrypted secret storage
│       │   │   ├── billing-engine.ts     # Usage tracking, splits
│       │   │   └── sandbox.ts            # Sandboxed agent testing
│       │   ├── connectors/
│       │   │   ├── interface.ts      # Base Connector interface
│       │   │   ├── a2a-orchestrator.ts   # A2A as a connector
│       │   │   ├── gmail.ts
│       │   │   ├── slack.ts
│       │   │   ├── shopify.ts
│       │   │   ├── hubspot.ts
```

```
│   │   │   ├── stripe-connector.ts
│   │   │   ├── notion.ts
│   │   │   ├── google-calendar.ts
│   │   │   ├── google-sheets.ts
│   │   │   ├── airtable.ts
│   │   │   └── webhook.ts
│   │   ├── queues/
│   │   │   ├── agent-execution.ts    # BullMQ agent task queue
│   │   │   ├── a2a-calls.ts          # A2A call queue
│   │   │   └── billing-events.ts     # Async billing processing
│   │   ├── middleware/
│   │   │   ├── auth.ts
│   │   │   ├── rate-limit.ts
│   │   │   └── tenant-scope.ts       # Ensures data isolation
│   │   └── index.ts
│   └── package.json
│
├── packages/
│   ├── shared/                   # Shared types, utils, constants
│   │   ├── types/
│   │   │   ├── agent.ts
│   │   │   ├── connector.ts
│   │   │   ├── marketplace.ts
│   │   │   ├── a2a.ts
│   │   │   └── billing.ts
│   │   └── utils/
│   └── config/                   # Shared ESLint, TS config
│
├── supabase/
│   ├── migrations/               # Ordered SQL migrations
│   └── seed.sql                  # Demo data for development
│
├── turbo.json
├── package.json
└── .env.example
```

4. Tech Stack

Frontend

Component	Technology	Rationale
Framework	Next.js 14 (App Router)	SSR, RSC, great DX
Language	TypeScript	Type safety across stack
Styling	Tailwind CSS + shadcn/ui	Rapid UI development with premium customization
State	Zustand	Lightweight, simple global state
Real-time	Socket.io client	Live agent activity, earnings updates
Charts	Recharts	Earnings dashboards, usage analytics
Fonts	Plus Jakarta Sans (headings) + DM Sans (body)	Premium typography

Backend

Component	Technology	Rationale
Framework	Hono	Lightweight, fast, edge-compatible
Language	TypeScript	Shared types with frontend
Queue	BullMQ + Redis	Reliable async agent execution
Real-time	Socket.io	Bidirectional event streaming
Validation	Zod	Runtime type checking

AI Layer

Component	Technology	Rationale
Primary LLM	Claude API (claude-opus-4-6-20250918)	Best reasoning + tool use for agent runtime
Planner LLM	Claude API (claude-sonnet-4-5-20250929)	Cost-effective for planning/assembly
Tool Protocol	MCP (Model Context Protocol)	Standard for agent-to-tool communication

Component	Technology	Rationale
Agent Protocol	A2A (Agent-to-Agent Protocol)	Standard for agent-to-agent communication
Vector DB	Pinecone	Agent discovery, semantic search
Embeddings	Voyage AI	Vectorizing agent capabilities

Database & Infrastructure

Component	Technology	Rationale
Database	Supabase (PostgreSQL)	Auth, RLS, real-time, managed
Auth	Supabase Auth	OAuth, email/password, SSO
Secrets	Supabase Vault (V1), AWS Secrets Manager (V2)	Encrypted credential storage
Cache	Redis (Upstash)	Rate limiting, session cache, BullMQ
Payments	Stripe + Stripe Connect	User billing + creator payouts
Hosting	Vercel (frontend) + Railway (backend)	Auto-scaling, easy deploys
CI/CD	GitHub Actions	Automated testing + deployment
Monitoring	Sentry (errors) + PostHog (analytics)	Observability + product analytics

5. Supabase Database Schema & RLS

Core Tables

sql

```sql
-- ============================================================
-- USERS & ORGANIZATIONS
-- ============================================================

-- Profiles extend Supabase auth.users
CREATE TABLE profiles (
  id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  display_name TEXT NOT NULL,
  avatar_url TEXT,
  bio TEXT,
  role TEXT NOT NULL DEFAULT 'user' CHECK (role IN ('user', 'creator', 'admin')),
  stripe_customer_id TEXT,
  stripe_connect_account_id TEXT,  -- For creators receiving payouts
  onboarding_completed BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Organizations (multi-tenancy for teams)
CREATE TABLE organizations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,
  slug TEXT UNIQUE NOT NULL,
  owner_id UUID REFERENCES profiles(id) ON DELETE CASCADE,
  plan TEXT DEFAULT 'free' CHECK (plan IN ('free', 'starter', 'pro', 'business', 'enterprise')),
  stripe_customer_id TEXT,
  stripe_subscription_id TEXT,
  current_period_start TIMESTAMPTZ DEFAULT NOW(),
  current_period_end TIMESTAMPTZ DEFAULT (NOW() + INTERVAL '30 days'),
  -- Usage tracking (reset each billing period)
  tasks_used_this_period INTEGER DEFAULT 0,
  a2a_calls_used_this_period INTEGER DEFAULT 0,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE org_members (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id UUID REFERENCES organizations(id) ON DELETE CASCADE,
  user_id UUID REFERENCES profiles(id) ON DELETE CASCADE,
  role TEXT DEFAULT 'member' CHECK (role IN ('owner', 'admin', 'member')),
  created_at TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(org_id, user_id)
```

```
);

-- ============================================================
-- CONNECTORS & CREDENTIALS
-- ============================================================

-- Available connector types (system-level)
CREATE TABLE connector_types (
  id TEXT PRIMARY KEY,  -- e.g., 'gmail', 'shopify', 'slack'
  name TEXT NOT NULL,
  description TEXT,
  icon_url TEXT,
  auth_type TEXT NOT NULL CHECK (auth_type IN ('oauth2', 'api_key', 'webhook')),
  oauth_config JSONB,  -- client_id, scopes, auth_url, token_url
  category TEXT NOT NULL,  -- 'communication', 'ecommerce', 'productivity', etc.
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- User's connected tools (one per connector per org)
CREATE TABLE connections (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id UUID REFERENCES organizations(id) ON DELETE CASCADE,
  connector_type_id TEXT REFERENCES connector_types(id),
  status TEXT DEFAULT 'active' CHECK (status IN ('active', 'expired', 'revoked', 'error')),
  -- Encrypted credentials stored in vault, referenced by ID
  credential_vault_id TEXT NOT NULL,
  scopes TEXT[],
  metadata JSONB,  -- Account name, email, shop URL, etc.
  connected_at TIMESTAMPTZ DEFAULT NOW(),
  expires_at TIMESTAMPTZ,
  last_used_at TIMESTAMPTZ,
  UNIQUE(org_id, connector_type_id)
);

-- ============================================================
-- AGENTS & MARKETPLACE
-- ============================================================

-- Agents (core entity — created by builders)
CREATE TABLE agents (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  creator_id UUID REFERENCES profiles(id) ON DELETE CASCADE,
```

```
-- Agent identity
name TEXT NOT NULL,
slug TEXT UNIQUE NOT NULL,
description TEXT NOT NULL,
long_description TEXT,  -- Markdown, shown on detail page
icon_url TEXT,
category TEXT NOT NULL,
tags TEXT[],

-- Agent configuration
system_prompt TEXT NOT NULL,
model TEXT DEFAULT 'claude-opus-4-6-20250918',
required_connectors TEXT[],  -- Which connector_types this agent needs
optional_connectors TEXT[],

-- A2A Agent Card (auto-generated, stored for fast lookups)
a2a_agent_card JSONB NOT NULL,
a2a_endpoint TEXT,  -- Platform-managed endpoint for this agent

-- Agent capabilities (for discovery and matching)
capabilities JSONB NOT NULL,  -- Structured: [{skill_id, name, description, input_schema, output_schema}]
capability_embeddings TEXT,  -- Pinecone vector ID for semantic search

-- Marketplace listing
status TEXT DEFAULT 'draft' CHECK (status IN ('draft', 'in_review', 'published', 'suspended', 'archived')),
visibility TEXT DEFAULT 'private' CHECK (visibility IN ('private', 'public', 'unlisted')),

-- Pricing
pricing_model TEXT DEFAULT 'monthly' CHECK (pricing_model IN ('free', 'monthly', 'per_use', 'tiered')),
price_cents INTEGER DEFAULT 0,  -- Monthly subscription price
per_use_price_cents INTEGER DEFAULT 0,  -- Per-execution price
a2a_call_price_cents INTEGER DEFAULT 1,  -- Price when called by another agent

-- Stats (denormalized for performance)
total_rentals INTEGER DEFAULT 0,
active_rentals INTEGER DEFAULT 0,
total_a2a_calls INTEGER DEFAULT 0,
avg_rating NUMERIC(3,2) DEFAULT 0,
review_count INTEGER DEFAULT 0,

-- Radar chart stats (auto-calculated, 0-99 each)
stats JSONB DEFAULT '{"speed": 50, "accuracy": 50, "reliability": 50, "popularity": 0, "versatility": 0}'::jsonb,
stats_updated_at TIMESTAMPTZ DEFAULT NOW(),
```

```
  -- Versioning
  version TEXT DEFAULT '1.0.0',
  changelog JSONB,

  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW(),
  published_at TIMESTAMPTZ
);

-- Agent reviews
CREATE TABLE agent_reviews (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  agent_id UUID REFERENCES agents(id) ON DELETE CASCADE,
  reviewer_id UUID REFERENCES profiles(id) ON DELETE CASCADE,
  rating INTEGER NOT NULL CHECK (rating >= 1 AND rating <= 5),
  title TEXT,
  body TEXT,
  helpful_count INTEGER DEFAULT 0,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(agent_id, reviewer_id)  -- One review per user per agent
);

-- ============================================================
-- RENTALS & USAGE
-- ============================================================

-- When a user/org subscribes to use an agent
CREATE TABLE rentals (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  agent_id UUID REFERENCES agents(id) ON DELETE CASCADE,
  renter_org_id UUID REFERENCES organizations(id) ON DELETE CASCADE,
  renter_user_id UUID REFERENCES profiles(id),

  status TEXT DEFAULT 'active' CHECK (status IN ('active', 'paused', 'cancelled', 'expired')),

  -- Billing
  stripe_subscription_id TEXT,
  current_period_start TIMESTAMPTZ,
  current_period_end TIMESTAMPTZ,
  monthly_price_cents INTEGER NOT NULL,

  -- Usage tracking
  total_executions INTEGER DEFAULT 0,
```

```
  total_a2a_calls_made INTEGER DEFAULT 0,
  total_a2a_calls_received INTEGER DEFAULT 0,

  -- Permissions (which of the org's connections this agent can use)
  allowed_connection_ids UUID[],

  started_at TIMESTAMPTZ DEFAULT NOW(),
  cancelled_at TIMESTAMPTZ,
  UNIQUE(agent_id, renter_org_id)  -- One rental per agent per org
);

-- Individual agent execution log
CREATE TABLE agent_executions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  rental_id UUID REFERENCES rentals(id) ON DELETE CASCADE,
  agent_id UUID REFERENCES agents(id),
  org_id UUID REFERENCES organizations(id),

  -- Task details
  input_text TEXT NOT NULL,  -- What the user asked
  status TEXT DEFAULT 'queued' CHECK (status IN ('queued', 'running', 'awaiting_approval', 'completed', 'failed', 'cancelled'

  -- Execution trace
  steps JSONB,  -- Array of {tool_name, input, output, duration_ms, a2a_call_id?}
  result JSONB,
  error TEXT,

  -- Metrics
  total_tokens INTEGER DEFAULT 0,
  total_tool_calls INTEGER DEFAULT 0,
  total_a2a_calls INTEGER DEFAULT 0,
  duration_ms INTEGER,
  cost_cents INTEGER DEFAULT 0,

  started_at TIMESTAMPTZ DEFAULT NOW(),
  completed_at TIMESTAMPTZ
);

-- ============================================================
-- A2A (AGENT-TO-AGENT) CALLS
-- ============================================================

CREATE TABLE a2a_calls (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
  -- Caller
  caller_agent_id UUID REFERENCES agents(id),
  caller_execution_id UUID REFERENCES agent_executions(id),
  caller_org_id UUID REFERENCES organizations(id),

  -- Target
  target_agent_id UUID REFERENCES agents(id),

  -- Task
  task_description TEXT NOT NULL,
  input_payload JSONB,
  output_payload JSONB,
  status TEXT DEFAULT 'pending' CHECK (status IN ('pending', 'running', 'completed', 'failed', 'timeout')),

  -- Billing
  cost_cents INTEGER NOT NULL,
  platform_fee_cents INTEGER NOT NULL,
  creator_earning_cents INTEGER NOT NULL,

  -- Metrics
  duration_ms INTEGER,

  created_at TIMESTAMPTZ DEFAULT NOW(),
  completed_at TIMESTAMPTZ
);

-- ============================================================
-- WORKFLOWS
-- ============================================================

-- A saved workflow = a team of agents configured to work together
CREATE TABLE workflows (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id UUID REFERENCES organizations(id) ON DELETE CASCADE,
  created_by UUID REFERENCES profiles(id),

  name TEXT NOT NULL,
  description TEXT,

  -- Which agents are in this workflow
  agent_ids UUID[] NOT NULL,

  -- Configuration
```

```
  trigger_type TEXT DEFAULT 'manual' CHECK (trigger_type IN ('manual', 'schedule', 'webhook', 'event')),
  trigger_config JSONB,

  -- Assembly metadata (if created by AI Workflow Assembler)
  assembled_by_ai BOOLEAN DEFAULT FALSE,
  assembly_prompt TEXT,  -- Original user description

  status TEXT DEFAULT 'active' CHECK (status IN ('active', 'paused', 'archived')),

  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- ============================================================
-- CREATOR EARNINGS
-- ============================================================

CREATE TABLE creator_earnings (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  creator_id UUID REFERENCES profiles(id) ON DELETE CASCADE,

  source TEXT NOT NULL CHECK (source IN ('rental', 'a2a_call', 'bonus')),
  source_id UUID,  -- References rental.id or a2a_call.id
  agent_id UUID REFERENCES agents(id),

  gross_amount_cents INTEGER NOT NULL,
  platform_fee_cents INTEGER NOT NULL,  -- 15% platform cut
  net_amount_cents INTEGER NOT NULL,

  -- Payout tracking
  payout_status TEXT DEFAULT 'pending' CHECK (payout_status IN ('pending', 'processing', 'paid', 'failed')),
  stripe_transfer_id TEXT,

  period_start TIMESTAMPTZ,
  period_end TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- ============================================================
-- APPROVAL QUEUE (for sensitive agent actions)
-- ============================================================

CREATE TABLE approval_queue (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
  execution_id UUID REFERENCES agent_executions(id) ON DELETE CASCADE,
  org_id UUID REFERENCES organizations(id),

  action_type TEXT NOT NULL,  -- e.g., 'send_email', 'process_refund', 'delete_record'
  action_details JSONB NOT NULL,

  status TEXT DEFAULT 'pending' CHECK (status IN ('pending', 'approved', 'rejected', 'expired')),
  reviewed_by UUID REFERENCES profiles(id),
  reviewed_at TIMESTAMPTZ,

  expires_at TIMESTAMPTZ DEFAULT (NOW() + INTERVAL '24 hours'),
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

Row-Level Security Policies

```
sql
```

```sql
-- Enable RLS on all tables
ALTER TABLE profiles ENABLE ROW LEVEL SECURITY;
ALTER TABLE organizations ENABLE ROW LEVEL SECURITY;
ALTER TABLE org_members ENABLE ROW LEVEL SECURITY;
ALTER TABLE connections ENABLE ROW LEVEL SECURITY;
ALTER TABLE agents ENABLE ROW LEVEL SECURITY;
ALTER TABLE agent_reviews ENABLE ROW LEVEL SECURITY;
ALTER TABLE rentals ENABLE ROW LEVEL SECURITY;
ALTER TABLE agent_executions ENABLE ROW LEVEL SECURITY;
ALTER TABLE a2a_calls ENABLE ROW LEVEL SECURITY;
ALTER TABLE workflows ENABLE ROW LEVEL SECURITY;
ALTER TABLE creator_earnings ENABLE ROW LEVEL SECURITY;
ALTER TABLE approval_queue ENABLE ROW LEVEL SECURITY;

-- Profiles: users can read any profile, edit only their own
CREATE POLICY "Public profiles are viewable" ON profiles FOR SELECT USING (true);
CREATE POLICY "Users can update own profile" ON profiles FOR UPDATE USING (auth.uid() = id);

-- Organizations: members can view, owners can edit
CREATE POLICY "Org members can view" ON organizations FOR SELECT
  USING (id IN (SELECT org_id FROM org_members WHERE user_id = auth.uid()));
CREATE POLICY "Org owners can update" ON organizations FOR UPDATE
  USING (owner_id = auth.uid());

-- Connections: only org members can view/manage their connections
CREATE POLICY "Org members can view connections" ON connections FOR SELECT
  USING (org_id IN (SELECT org_id FROM org_members WHERE user_id = auth.uid()));
CREATE POLICY "Org admins can manage connections" ON connections FOR ALL
  USING (org_id IN (SELECT org_id FROM org_members WHERE user_id = auth.uid() AND role IN ('owner', 'admin')));

-- Agents: published agents are public, drafts only visible to creator
CREATE POLICY "Published agents are public" ON agents FOR SELECT
  USING (status = 'published' AND visibility = 'public');
CREATE POLICY "Creators can manage own agents" ON agents FOR ALL
  USING (creator_id = auth.uid());

-- Rentals: only the renting org can see their rentals
CREATE POLICY "Renters can view own rentals" ON rentals FOR SELECT
  USING (renter_org_id IN (SELECT org_id FROM org_members WHERE user_id = auth.uid()));

-- Agent executions: only the org that triggered can view
CREATE POLICY "Org members can view executions" ON agent_executions FOR SELECT
  USING (org_id IN (SELECT org_id FROM org_members WHERE user_id = auth.uid()));
```

```sql
-- Creator earnings: only the creator can see their earnings
CREATE POLICY "Creators can view own earnings" ON creator_earnings FOR SELECT
  USING (creator_id = auth.uid());

-- Approval queue: org members can view/act on their approvals
CREATE POLICY "Org members can manage approvals" ON approval_queue FOR ALL
  USING (org_id IN (SELECT org_id FROM org_members WHERE user_id = auth.uid()));
```

Indexes

```sql
-- Marketplace search & browse
CREATE INDEX idx_agents_status_category ON agents(status, category) WHERE status = 'published';
CREATE INDEX idx_agents_tags ON agents USING GIN(tags);
CREATE INDEX idx_agents_avg_rating ON agents(avg_rating DESC) WHERE status = 'published';
CREATE INDEX idx_agents_total_rentals ON agents(total_rentals DESC) WHERE status = 'published';
CREATE INDEX idx_agents_creator ON agents(creator_id);
CREATE INDEX idx_agents_slug ON agents(slug);

-- Rental lookups
CREATE INDEX idx_rentals_org ON rentals(renter_org_id) WHERE status = 'active';
CREATE INDEX idx_rentals_agent ON rentals(agent_id) WHERE status = 'active';

-- A2A call lookups
CREATE INDEX idx_a2a_calls_caller ON a2a_calls(caller_agent_id);
CREATE INDEX idx_a2a_calls_target ON a2a_calls(target_agent_id);
CREATE INDEX idx_a2a_calls_execution ON a2a_calls(caller_execution_id);

-- Earnings
CREATE INDEX idx_earnings_creator ON creator_earnings(creator_id, created_at DESC);
CREATE INDEX idx_earnings_payout ON creator_earnings(payout_status) WHERE payout_status = 'pending';

-- Executions
CREATE INDEX idx_executions_rental ON agent_executions(rental_id);
CREATE INDEX idx_executions_status ON agent_executions(status) WHERE status IN ('queued', 'running');
```

---

6. Authentication & Multi-Tenancy

Auth Flow

1. User signs up via Supabase Auth (email/password or Google OAuth)
2. On first login, a `profiles` row is created via database trigger
3. User creates or joins an `organization` (auto-created "Personal" org on signup)
4. All subsequent actions are scoped to the user's active organization
5. Creators opt-in via "Become a Creator" flow which triggers Stripe Connect onboarding

Multi-Tenancy Model

- Every data query includes `org_id` scoping via RLS
- Connections (OAuth tokens) belong to orgs, not users
- Rentals belong to orgs — any org member can use rented agents
- Agent executions are logged per-org for billing and audit
- A creator's agent code/prompts are never exposed to renters

---

7. Connector Interface & OAuth Integration Layer

Base Connector Interface

typescript

```typescript
// packages/shared/types/connector.ts

export interface MCPToolDefinition {
  name: string;
  description: string;
  input_schema: {
    type: 'object';
    properties: Record<string, any>;
    required?: string[];
  };
}

export interface ToolResult {
  success: boolean;
  data?: any;
  error?: string;
}

export interface Connector {
  id: string;                    // e.g., 'gmail', 'shopify'
  name: string;
  description: string;
  authType: 'oauth2' | 'api_key' | 'webhook';
  category: string;
  icon: string;

  // Authentication
  getOAuthURL(orgId: string, redirectUri: string): string;
  handleOAuthCallback(code: string, orgId: string): Promise<{ credentialVaultId: string }>;
  refreshToken(credentialVaultId: string): Promise<void>;
  testConnection(credentialVaultId: string): Promise<boolean>;

  // MCP Tool Definitions
  getTools(): MCPToolDefinition[];

  // Execution (called by agent runtime)
  executeTool(
    toolName: string,
    params: Record<string, any>,
    credentialVaultId: string
  ): Promise<ToolResult>;
}
```

One-Click OAuth Flow

1. User clicks "Connect Shopify" on the connections page
2. Frontend redirects to `/api/connections/oauth/shopify/start?org_id=xxx`
3. Backend generates OAuth URL via connector's `getOAuthURL()`, redirects user
4. User authorizes on Shopify
5. Shopify redirects back to `/api/connections/oauth/shopify/callback?code=xxx`
6. Backend calls connector's `handleOAuthCallback()` — encrypts tokens, stores in vault
7. Creates `connections` row with status 'active'
8. Redirects user back to connections page with success toast

MVP Connectors (10)

Connector	Auth Type	Category	Key Tools
Gmail	OAuth2	Communication	send_email, read_emails, search_emails
Slack	OAuth2	Communication	send_message, read_channel, create_channel
Shopify	OAuth2	E-commerce	get_orders, process_refund, update_inventory
HubSpot	OAuth2	CRM	create_contact, update_deal, log_activity
Stripe	API Key	Payments	create_invoice, get_payments, issue_refund
Notion	OAuth2	Productivity	create_page, query_database, update_block
Google Calendar	OAuth2	Productivity	create_event, get_events, update_event
Google Sheets	OAuth2	Data	read_sheet, write_sheet, create_sheet
Airtable	API Key	Data	query_records, create_record, update_record
Webhooks	API Key	Custom	send_webhook, register_listener

8. Agent Runtime — Agentic Loop

typescript

```typescript
// apps/api/src/services/agent-runtime.ts

import Anthropic from '@anthropic-ai/sdk';

const anthropic = new Anthropic();

interface ExecutionContext {
  executionId: string;
  agent: Agent;              // From agents table
  rental: Rental;            // From rentals table
  orgId: string;
  connections: Connection[];  // Org's connected tools
  userInput: string;
}

export async function executeAgent(ctx: ExecutionContext): Promise<void> {
  const { agent, rental, orgId, connections, userInput } = ctx;

  // 1. Gather available tools from connected services
  const connectorTools = await gatherConnectorTools(
    agent.required_connectors,
    connections,
    rental.allowed_connection_ids
  );

  // 2. Add A2A orchestrator as a tool (if agent has permission)
  const a2aTools = getA2AOrchestratorTools();

  // 3. Combine all tools
  const allTools: Anthropic.Tool[] = [...connectorTools, ...a2aTools];

  // 4. Build message history
  const messages: Anthropic.MessageParam[] = [
    { role: 'user', content: userInput }
  ];

  // 5. Agentic loop
  let iterationCount = 0;
  const MAX_ITERATIONS = 25;

  while (iterationCount < MAX_ITERATIONS) {
    iterationCount++;
```

```
const response = await anthropic.messages.create({
  model: agent.model,
  max_tokens: 4096,
  system: agent.system_prompt,
  tools: allTools,
  messages,
});

// Log the step
await logExecutionStep(ctx.executionId, response);

// Emit real-time update
emitAgentActivity(orgId, ctx.executionId, response);

// If no tool use, agent is done
if (response.stop_reason === 'end_turn') {
  const finalText = response.content
    .filter(b => b.type === 'text')
    .map(b => b.text)
    .join('\n');

  await completeExecution(ctx.executionId, { result: finalText });
  return;
}

// Process tool calls
if (response.stop_reason === 'tool_use') {
  const toolBlocks = response.content.filter(b => b.type === 'tool_use');
  const toolResults: Anthropic.ToolResultBlockParam[] = [];

  for (const toolCall of toolBlocks) {
    // Check if action needs approval
    if (requiresApproval(toolCall.name, toolCall.input)) {
      await pauseForApproval(ctx, toolCall);
      return;  // Execution resumes after approval
    }

    let result: ToolResult;

    // Route to correct handler
    if (toolCall.name === 'delegate_to_agent' || toolCall.name === 'discover_agents') {
      // A2A call
      result = await handleA2AToolCall(toolCall, ctx);
    } else {
```

```
        // Regular connector tool call
        result = await handleConnectorToolCall(toolCall, connections, rental);
      }

      toolResults.push({
        type: 'tool_result',
        tool_use_id: toolCall.id,
        content: JSON.stringify(result),
      });
    }

    // Add assistant response + tool results to history
    messages.push({ role: 'assistant', content: response.content });
    messages.push({ role: 'user', content: toolResults });
  }
}

// Max iterations reached
await completeExecution(ctx.executionId, {
  error: 'Agent reached maximum iteration limit'
});
}

// Gather MCP tools from the org's connected services
async function gatherConnectorTools(
  requiredConnectors: string[],
  orgConnections: Connection[],
  allowedConnectionIds: string[]
): Promise<Anthropic.Tool[]> {
  const tools: Anthropic.Tool[] = [];

  for (const connectorId of requiredConnectors) {
    const connection = orgConnections.find(
      c => c.connector_type_id === connectorId &&
           allowedConnectionIds.includes(c.id)
    );

    if (!connection) continue;

    const connector = loadConnector(connectorId);
    const connectorTools = connector.getTools();

    for (const tool of connectorTools) {
      tools.push({
```

```
      name: `${connectorId}__${tool.name}`,  // Namespace: shopify__get_orders
      description: tool.description,
      input_schema: tool.input_schema,
    });
  }
 }

 return tools;
}
```

---

9. A2A Orchestration Layer

A2A Agent Card (Auto-Generated)

When a creator publishes an agent, the platform auto-generates an A2A-compliant Agent Card:

typescript

```typescript
// apps/api/src/services/agent-registry.ts

export interface A2AAgentCard {
  name: string;
  description: string;
  url: string;              // Platform-managed endpoint
  version: string;

  capabilities: {
    streaming: boolean;
    pushNotifications: boolean;
  };

  skills: Array<{
    id: string;
    name: string;
    description: string;
    tags: string[];
    inputSchema: JSONSchema;
    outputSchema: JSONSchema;
  }>;

  authentication: {
    schemes: string[];      // ['platform_token']
  };

  pricing: {
    perCallCents: number;
  };

  // Platform metadata
  agentId: string;
  creatorId: string;
  rating: number;
  totalCalls: number;
}

export function generateAgentCard(agent: Agent): A2AAgentCard {
  return {
    name: agent.name,
    description: agent.description,
    url: `https://api.agentspark.io/a2a/agents/${agent.slug}`,
    version: agent.version,
```

```
    capabilities: { streaming: true, pushNotifications: false },
    skills: agent.capabilities.map(cap => ({
      id: cap.skill_id,
      name: cap.name,
      description: cap.description,
      tags: agent.tags,
      inputSchema: cap.input_schema,
      outputSchema: cap.output_schema,
    })),
    authentication: { schemes: ['platform_token'] },
    pricing: { perCallCents: agent.a2a_call_price_cents },
    agentId: agent.id,
    creatorId: agent.creator_id,
    rating: agent.avg_rating,
    totalCalls: agent.total_a2a_calls,
  };
}
```

A2A Orchestrator Connector

typescript

```
// apps/api/src/connectors/a2a-orchestrator.ts
// This is a special connector that lets agents call other agents.
// It follows the EXACT same Connector interface as Gmail, Shopify, etc.

import { Connector, MCPToolDefinition, ToolResult } from '@agentspark/shared';

export const a2aOrchestratorConnector: Connector = {
  id: 'a2a-orchestrator',
  name: 'Agent Network',
  description: 'Delegate tasks to other specialized agents on the Agent Spark marketplace',
  authType: 'platform',  // Uses platform-level auth, not user OAuth
  category: 'orchestration',
  icon: '/icons/network.svg',

  getTools(): MCPToolDefinition[] {
    return [
      {
        name: 'discover_agents',
        description: 'Search the Agent Spark marketplace for agents with specific capabilities. Returns a list of matching agents v
        input_schema: {
          type: 'object',
          properties: {
            capability_needed: {
              type: 'string',
              description: 'Describe what you need another agent to do, e.g., "process Shopify refunds" or "analyze spreadsheet dat
            },
            max_results: {
              type: 'number',
              description: 'Maximum number of agents to return (default: 5)',
            },
          },
          required: ['capability_needed'],
        },
      },
      {
        name: 'delegate_to_agent',
        description: 'Send a task to another agent on the platform. The target agent will execute the task using its own capabilitie
        input_schema: {
          type: 'object',
          properties: {
            target_agent_id: {
              type: 'string',
              description: 'The ID of the agent to delegate to (from discover_agents results)',
```

```
        },
        task_description: {
          type: 'string',
          description: 'Clear description of what you need the target agent to do',
        },
        context: {
          type: 'object',
          description: 'Any relevant data the target agent needs to complete the task',
        },
      },
      required: ['target_agent_id', 'task_description'],
    },
  },
 ];
},

async executeTool(
  toolName: string,
  params: Record<string, any>,
  _credentialVaultId: string,
  executionContext?: { executionId: string; orgId: string; agentId: string }
): Promise<ToolResult> {

  if (toolName === 'discover_agents') {
    // 1. Semantic search against agent capability embeddings
    const embedding = await generateEmbedding(params.capability_needed);
    const matches = await pinecone.query({
      vector: embedding,
      topK: params.max_results || 5,
      filter: { status: 'published' },
    });

    // 2. Fetch full agent cards
    const agents = await fetchAgentCards(matches.map(m => m.id));

    return {
      success: true,
      data: agents.map(a => ({
        agent_id: a.agentId,
        name: a.name,
        description: a.description,
        skills: a.skills.map(s => s.name),
        rating: a.rating,
        price_per_call_cents: a.pricing.perCallCents,
```

```
    })),
  };
}

if (toolName === 'delegate_to_agent') {
  const { target_agent_id, task_description, context } = params;

  // 1. Create A2A call record
  const a2aCall = await createA2ACall({
    callerAgentId: executionContext.agentId,
    callerExecutionId: executionContext.executionId,
    callerOrgId: executionContext.orgId,
    targetAgentId: target_agent_id,
    taskDescription: task_description,
    inputPayload: context,
  });

  // 2. Execute the target agent with the task
  const targetAgent = await fetchAgent(target_agent_id);
  const result = await executeA2ATask(targetAgent, {
    task: task_description,
    context,
    callerOrgId: executionContext.orgId,
  });

  // 3. Record earnings
  const cost = targetAgent.a2a_call_price_cents;
  const platformFee = Math.round(cost * 0.15);
  const creatorEarning = cost - platformFee;

  await recordEarning({
    creatorId: targetAgent.creator_id,
    source: 'a2a_call',
    sourceId: a2aCall.id,
    agentId: target_agent_id,
    grossAmountCents: cost,
    platformFeeCents: platformFee,
    netAmountCents: creatorEarning,
  });

  // 4. Update A2A call record
  await completeA2ACall(a2aCall.id, {
    outputPayload: result,
    status: 'completed',
```

```
      durationMs: Date.now() - a2aCall.createdAt,
    });

    // 5. Emit real-time event (for network visualizer)
    emitA2ACall(executionContext.orgId, {
      from: executionContext.agentId,
      to: target_agent_id,
      task: task_description,
      result: result,
    });

    return {
      success: true,
      data: result,
    };
  }

  return { success: false, error: `Unknown tool: ${toolName}` };
 },
};
```

---

10. Marketplace Engine

Search & Discovery

typescript

```typescript
// apps/api/src/services/marketplace.ts

interface MarketplaceSearchParams {
  query?: string;          // Text search
  category?: string;
  tags?: string[];
  priceRange?: { min: number; max: number };
  minRating?: number;
  requiredConnectors?: string[];
  sortBy?: 'popular' | 'rating' | 'newest' | 'price_low' | 'price_high';
  page?: number;
  pageSize?: number;
}

export async function searchMarketplace(params: MarketplaceSearchParams) {
  let query = supabase
    .from('agents')
    .select('*, profiles!creator_id(display_name, avatar_url)')
    .eq('status', 'published')
    .eq('visibility', 'public');

  // Category filter
  if (params.category) {
    query = query.eq('category', params.category);
  }

  // Tag filter
  if (params.tags?.length) {
    query = query.overlaps('tags', params.tags);
  }

  // Price range
  if (params.priceRange) {
    query = query
      .gte('price_cents', params.priceRange.min)
      .lte('price_cents', params.priceRange.max);
  }

  // Rating filter
  if (params.minRating) {
    query = query.gte('avg_rating', params.minRating);
  }
```

```
  // Required connectors
  if (params.requiredConnectors?.length) {
    query = query.contains('required_connectors', params.requiredConnectors);
  }

  // Sort
  switch (params.sortBy) {
    case 'popular': query = query.order('total_rentals', { ascending: false }); break;
    case 'rating': query = query.order('avg_rating', { ascending: false }); break;
    case 'newest': query = query.order('published_at', { ascending: false }); break;
    case 'price_low': query = query.order('price_cents', { ascending: true }); break;
    case 'price_high': query = query.order('price_cents', { ascending: false }); break;
    default: query = query.order('total_rentals', { ascending: false });
  }

  // Pagination
  const page = params.page || 1;
  const pageSize = params.pageSize || 24;
  query = query.range((page - 1) * pageSize, page * pageSize - 1);

  // If text query, also do semantic search for better results
  if (params.query) {
    const semanticResults = await semanticAgentSearch(params.query);
    // Merge and rank results (semantic + structured)
  }

  return query;
}
```

Agent Categories

typescript

```typescript
export const AGENT_CATEGORIES = [
  { id: 'customer-support', name: 'Customer Support', icon: 'headphones' },
  { id: 'sales', name: 'Sales & CRM', icon: 'trending-up' },
  { id: 'marketing', name: 'Marketing', icon: 'megaphone' },
  { id: 'ecommerce', name: 'E-commerce', icon: 'shopping-cart' },
  { id: 'finance', name: 'Finance & Accounting', icon: 'dollar-sign' },
  { id: 'hr', name: 'HR & Recruiting', icon: 'users' },
  { id: 'productivity', name: 'Productivity', icon: 'zap' },
  { id: 'data', name: 'Data & Analytics', icon: 'bar-chart' },
  { id: 'development', name: 'Development', icon: 'code' },
  { id: 'content', name: 'Content Creation', icon: 'pen-tool' },
  { id: 'operations', name: 'Operations', icon: 'settings' },
  { id: 'utility', name: 'Utility Agents', icon: 'wrench' },
] as const;
```

---

11. AI Workflow Assembler

The Workflow Assembler is the "describe what you need" entry point. It uses Claude to understand the user's request and match it to agents on the marketplace.

typescript

```
// apps/api/src/services/workflow-assembler.ts

import Anthropic from '@anthropic-ai/sdk';

const anthropic = new Anthropic();

interface AssemblyResult {
  recommended_agents: Array<{
    agent: Agent;
    reason: string;          // Why this agent was selected
    role_in_workflow: string;  // What it does in the team
  }>;
  workflow_description: string;
  required_connections: string[];  // Which OAuth connections user needs
  estimated_monthly_cost_cents: number;
}

export async function assembleWorkflow(
  userPrompt: string,
  orgConnections: Connection[]  // What the user already has connected
): Promise<AssemblyResult> {

  // 1. Use Claude to extract intent and required capabilities
  const analysisResponse = await anthropic.messages.create({
    model: 'claude-sonnet-4-5-20250929',
    max_tokens: 2048,
    system: `You are the Agent Spark Workflow Assembler. Given a user's description of what they need automated, analyze th

    1. "capabilities_needed": Array of specific capabilities needed (e.g., "process shopify refunds", "send customer emails", "up
    2. "connectors_needed": Array of service connectors needed (e.g., "shopify", "gmail", "google-sheets")
    3. "workflow_summary": Plain English description of how agents should work together
    4. "priority_order": Which capability is the primary one vs supporting

    Respond ONLY with valid JSON. No markdown, no backticks.`,
    messages: [{ role: 'user', content: userPrompt }],
  });

  const analysis = JSON.parse(
    analysisResponse.content[0].type === 'text' ? analysisResponse.content[0].text : '{}'
  );

  // 2. For each capability needed, search the marketplace
  const agentCandidates: Agent[] = [];
```

```
for (const capability of analysis.capabilities_needed) {
  const results = await searchMarketplace({
    query: capability,
    sortBy: 'rating',
    pageSize: 5,
  });
  agentCandidates.push(...results.data);
}

// 3. Deduplicate and rank
const uniqueAgents = deduplicateAgents(agentCandidates);

// 4. Use Claude to select the best team from candidates
const selectionResponse = await anthropic.messages.create({
  model: 'claude-sonnet-4-5-20250929',
  max_tokens: 2048,
  system: `You are selecting the best team of AI agents for a user's workflow. Given the user's request and available agents, se

  Respond with JSON: { "selected": [{ "agent_id": "...", "reason": "...", "role": "..." }] }`,
  messages: [{
    role: 'user',
    content: JSON.stringify({
      user_request: userPrompt,
      available_agents: uniqueAgents.map(a => ({
        id: a.id,
        name: a.name,
        description: a.description,
        capabilities: a.capabilities,
        rating: a.avg_rating,
        price_cents: a.price_cents,
        required_connectors: a.required_connectors,
      })),
    }),
  }],
});

const selection = JSON.parse(
  selectionResponse.content[0].type === 'text' ? selectionResponse.content[0].text : '{"selected":[]}'
);

// 5. Build the recommendation
const recommendedAgents = selection.selected.map(s => {
  const agent = uniqueAgents.find(a => a.id === s.agent_id);
```

```
    return { agent, reason: s.reason, role_in_workflow: s.role };
  }).filter(Boolean);

  // 6. Calculate what connections are needed
  const allRequiredConnectors = [...new Set(
    recommendedAgents.flatMap(ra => ra.agent.required_connectors)
  )];
  const existingConnectors = orgConnections.map(c => c.connector_type_id);
  const missingConnectors = allRequiredConnectors.filter(
    c => !existingConnectors.includes(c)
  );

  return {
    recommended_agents: recommendedAgents,
    workflow_description: analysis.workflow_summary,
    required_connections: missingConnectors,
    estimated_monthly_cost_cents: recommendedAgents.reduce(
      (sum, ra) => sum + ra.agent.price_cents, 0
    ),
  };
}
```

---

12. Billing, Pricing & Cost Optimization

Revenue Model — Two Streams

Agent Spark has two distinct revenue streams that ensure profitability:

Stream 1 — Platform Subscriptions (covers compute costs + margin) Users pay a monthly subscription to the platform for agent execution capacity:

Plan	Price	Tasks/Month	A2A Calls Included	Target User
Free	$0	50	10	Trial/exploration
Starter	$29/mo	500	100	Small business getting started
Pro	$79/mo	2,000	500	Growing business, multiple workflows
Business	$199/mo	10,000	2,500	Scaled operations
Enterprise	Custom	Unlimited	Unlimited	Large organizations

A "task" = one agent execution (the full agentic loop from user input to completion). An "A2A call" = one agent delegating to another agent. Overage is billed at $0.03/task and $0.05/A2A call.

Stream 2 — Marketplace Cut (pure margin) When a user rents an agent from a creator, the platform takes 15%:

- Agent rental: $49/mo → Creator gets $41.65, Platform gets $7.35
- A2A call revenue: if a creator prices their agent at $0.05/A2A call → Creator gets $0.0425, Platform gets $0.0075

Stripe Implementation

typescript

```
// apps/api/src/services/billing-engine.ts

// ============================================================
// PLATFORM SUBSCRIPTIONS
// ============================================================

const PLATFORM_PLANS = {
  free: {
    name: 'Free',
    stripe_price_id: null,
    monthly_tasks: 50,
    monthly_a2a_calls: 10,
    price_cents: 0,
  },
  starter: {
    name: 'Starter',
    stripe_price_id: process.env.STRIPE_STARTER_PRICE_ID,
    monthly_tasks: 500,
    monthly_a2a_calls: 100,
    price_cents: 2900,
  },
  pro: {
    name: 'Pro',
    stripe_price_id: process.env.STRIPE_PRO_PRICE_ID,
    monthly_tasks: 2000,
    monthly_a2a_calls: 500,
    price_cents: 7900,
  },
  business: {
    name: 'Business',
    stripe_price_id: process.env.STRIPE_BUSINESS_PRICE_ID,
    monthly_tasks: 10000,
    monthly_a2a_calls: 2500,
    price_cents: 19900,
  },
} as const;

// Check usage before every agent execution
export async function checkUsageAllowance(orgId: string): Promise<{
  allowed: boolean;
  remaining_tasks: number;
  remaining_a2a_calls: number;
  plan: string;
```

```
}> {
  const org = await fetchOrg(orgId);
  const plan = PLATFORM_PLANS[org.plan];

  // Count usage this billing period
  const { task_count, a2a_count } = await supabase.rpc('get_current_period_usage', {
    org_uuid: orgId,
    period_start: org.current_period_start,
  });

  return {
    allowed: task_count < plan.monthly_tasks,
    remaining_tasks: Math.max(0, plan.monthly_tasks - task_count),
    remaining_a2a_calls: Math.max(0, plan.monthly_a2a_calls - a2a_count),
    plan: org.plan,
  };
}

// ============================================================
// AGENT RENTALS (Creator Revenue)
// ============================================================

const PLATFORM_FEE_PERCENTAGE = 0.15;  // 15% platform cut

export async function createRental(
  orgId: string,
  agentId: string,
  userId: string,
  allowedConnectionIds: string[]
): Promise<Rental> {
  const agent = await fetchAgent(agentId);
  const org = await fetchOrg(orgId);

  // Free agents don't need Stripe
  if (agent.pricing_model === 'free') {
    return await createFreeRental(orgId, agentId, userId, allowedConnectionIds);
  }

  // Create Stripe subscription with application fee for platform cut
  const subscription = await stripe.subscriptions.create({
    customer: org.stripe_customer_id,
    items: [{ price: agent.stripe_price_id }],
    application_fee_percent: PLATFORM_FEE_PERCENTAGE * 100,  // 15%
    transfer_data: {
```

```
      destination: agent.creator_stripe_connect_id,  // Creator receives 85%
    },
    metadata: {
      org_id: orgId,
      agent_id: agentId,
      platform: 'agentspark',
    },
  });

  const rental = await supabase.from('rentals').insert({
    agent_id: agentId,
    renter_org_id: orgId,
    renter_user_id: userId,
    status: 'active',
    stripe_subscription_id: subscription.id,
    current_period_start: new Date(subscription.current_period_start * 1000),
    current_period_end: new Date(subscription.current_period_end * 1000),
    monthly_price_cents: agent.price_cents,
    allowed_connection_ids: allowedConnectionIds,
  }).select().single();

  await supabase.rpc('increment_agent_rentals', { agent_id: agentId });

  return rental.data;
}

// ============================================================
// CREATOR PAYOUTS (Stripe Connect)
// ============================================================

export async function onboardCreator(userId: string): Promise<string> {
  const account = await stripe.accounts.create({
    type: 'express',
    metadata: { user_id: userId, platform: 'agentspark' },
  });

  await supabase.from('profiles').update({
    stripe_connect_account_id: account.id,
    role: 'creator',
  }).eq('id', userId);

  const link = await stripe.accountLinks.create({
    account: account.id,
    refresh_url: `${BASE_URL}/creator/onboarding?refresh=true`,
```

```
    return_url: `${BASE_URL}/creator/earnings`,
    type: 'account_onboarding',
  });

  return link.url;
}

// Monthly payout processing for A2A call earnings
// (Rental payouts are handled automatically by Stripe Connect transfers)
export async function processA2APayouts(): Promise<void> {
  const pendingEarnings = await supabase
    .from('creator_earnings')
    .select('creator_id, SUM(net_amount_cents) as total')
    .eq('payout_status', 'pending')
    .eq('source', 'a2a_call')
    .group('creator_id');

  for (const earning of pendingEarnings.data) {
    const creator = await fetchProfile(earning.creator_id);
    if (!creator.stripe_connect_account_id) continue;
    if (earning.total < 1000) continue;  // Min $10 payout

    const transfer = await stripe.transfers.create({
      amount: earning.total,
      currency: 'usd',
      destination: creator.stripe_connect_account_id,
      metadata: { creator_id: earning.creator_id },
    });

    await supabase.from('creator_earnings')
      .update({ payout_status: 'paid', stripe_transfer_id: transfer.id })
      .eq('creator_id', earning.creator_id)
      .eq('payout_status', 'pending')
      .eq('source', 'a2a_call');
  }
}
```

Smart Model Routing (Cost Optimization)

Not every task needs the most expensive model. The platform auto-routes to the cheapest model that can handle the task, cutting average compute cost by 40-60%.

typescript

```typescript
// apps/api/src/services/model-router.ts

// Current pricing (per million tokens):
// Haiku 4.5:  $1 input  / $5 output   → ~$0.03 per task
// Sonnet 4.5: $3 input  / $15 output  → ~$0.10 per task
// Opus 4.5:   $5 input  / $25 output  → ~$0.17 per task

type ModelTier = 'haiku' | 'sonnet' | 'opus';

interface RoutingDecision {
  model: string;
  tier: ModelTier;
  estimated_cost_cents: number;
}

const MODEL_MAP = {
  haiku: 'claude-haiku-4-5-20251001',
  sonnet: 'claude-sonnet-4-5-20250929',
  opus: 'claude-opus-4-6-20250918',
} as const;

export function routeToModel(task: {
  tool_count: number;       // How many tools available
  input_length: number;     // Token estimate of input
  task_type: string;        // 'discovery', 'simple_action', 'multi_step', 'complex_reasoning'
  agent_model_preference?: string;  // Creator can set minimum model tier
}): RoutingDecision {

  // A2A discovery calls → always Haiku (just searching a database)
  if (task.task_type === 'discovery') {
    return { model: MODEL_MAP.haiku, tier: 'haiku', estimated_cost_cents: 3 };
  }

  // Simple single-tool actions → Haiku
  // e.g., "send this email", "look up this order"
  if (task.task_type === 'simple_action' && task.tool_count <= 3) {
    return { model: MODEL_MAP.haiku, tier: 'haiku', estimated_cost_cents: 3 };
  }

  // Multi-step workflows with several tools → Sonnet (default workhorse)
  // e.g., "find the order, process the refund, email the customer"
  if (task.task_type === 'multi_step' || task.tool_count <= 8) {
    return { model: MODEL_MAP.sonnet, tier: 'sonnet', estimated_cost_cents: 10 };
```

```
  }

  // Complex reasoning, many tools, or creator requested Opus → Opus
  // e.g., "analyze our sales data, identify trends, create a strategy"
  if (task.task_type === 'complex_reasoning' || task.tool_count > 8) {
    return { model: MODEL_MAP.opus, tier: 'opus', estimated_cost_cents: 17 };
  }

  // Respect creator's minimum model preference
  if (task.agent_model_preference) {
    const tier = task.agent_model_preference as ModelTier;
    return {
      model: MODEL_MAP[tier],
      tier,
      estimated_cost_cents: tier === 'haiku' ? 3 : tier === 'sonnet' ? 10 : 17
    };
  }

  // Default to Sonnet
  return { model: MODEL_MAP.sonnet, tier: 'sonnet', estimated_cost_cents: 10 };
}

// Classification call to determine task complexity (uses Haiku — costs $0.003)
export async function classifyTaskComplexity(
  userInput: string,
  availableTools: string[]
): Promise<string> {
  const response = await anthropic.messages.create({
    model: MODEL_MAP.haiku,
    max_tokens: 50,
    system: `Classify this task into exactly one category. Respond with ONLY the category name, nothing else.
Categories:
- discovery (searching/finding information)
- simple_action (one clear action to take)
- multi_step (2-5 steps needed)
- complex_reasoning (analysis, strategy, many steps)`,
    messages: [{ role: 'user', content: `Task: ${userInput}\nAvailable tools: ${availableTools.join(', ')}` }],
  });

  return response.content[0].type === 'text' ? response.content[0].text.trim() : 'multi_step';
}
```

Prompt Caching Strategy

Every agent has a fixed system prompt that gets sent on every execution. Cache it to cut repeat costs by 90%.

typescript

```
// apps/api/src/services/agent-runtime.ts (addition to executeAgent)

// When building the API call, use cache_control on the system prompt
const response = await anthropic.messages.create({
  model: routingDecision.model,
  max_tokens: 4096,
  system: [
    {
      type: 'text',
      text: agent.system_prompt,
      cache_control: { type: 'ephemeral' },  // Cache this block
    }
  ],
  tools: allTools,
  messages,
});

// First call: full price on system prompt tokens
// Subsequent calls (within 5 min TTL): 90% cheaper on cached tokens
// For a popular agent with 100 calls/hour, this saves ~$50-100/day
```

Batch Processing for Scheduled Tasks

typescript

```
// apps/api/src/queues/scheduled-agents.ts

// Scheduled agents (daily reports, weekly summaries) use the Batch API
// for 50% cost reduction since they don't need real-time responses

export async function queueScheduledExecution(
  rental: Rental,
  task: string
): Promise<void> {
  // Add to batch queue instead of executing immediately
  await batchQueue.add('scheduled-execution', {
    rental_id: rental.id,
    agent_id: rental.agent_id,
    org_id: rental.renter_org_id,
    task,
    use_batch_api: true,  // 50% discount
  });
}
```

Execution Guardrails (Prevent Runaway Costs)

typescript

```ts
// apps/api/src/services/agent-runtime.ts (guardrails)

const EXECUTION_LIMITS = {
  free: { max_iterations: 5, max_tokens_per_run: 10000, max_a2a_calls: 1 },
  starter: { max_iterations: 10, max_tokens_per_run: 25000, max_a2a_calls: 3 },
  pro: { max_iterations: 15, max_tokens_per_run: 50000, max_a2a_calls: 5 },
  business: { max_iterations: 25, max_tokens_per_run: 100000, max_a2a_calls: 10 },
} as const;

// In the agentic loop, enforce limits:
function checkGuardrails(
  plan: string,
  currentIteration: number,
  totalTokens: number,
  a2aCalls: number
): { allowed: boolean; reason?: string } {
  const limits = EXECUTION_LIMITS[plan];

  if (currentIteration >= limits.max_iterations) {
    return { allowed: false, reason: 'Maximum steps reached for your plan. Upgrade for longer workflows.' };
  }
  if (totalTokens >= limits.max_tokens_per_run) {
    return { allowed: false, reason: 'Token limit reached for this execution.' };
  }
  if (a2aCalls >= limits.max_a2a_calls) {
    return { allowed: false, reason: 'A2A call limit reached. Upgrade for more agent collaboration.' };
  }

  return { allowed: true };
}
```

Profitability Model at Scale

Unit economics at 1,000 paying users (avg Pro plan):

```
REVENUE
├── Platform subscriptions: 1,000 × $79 avg     = $79,000/mo
├── Marketplace cut (15% of agent rentals):
│   └── 1,000 users × 2 agents × $39 avg × 15% = $11,700/mo
├── A2A call revenue (15% cut)                  = ~$2,000/mo
└── Overage charges                             = ~$3,000/mo
                                    TOTAL  = ~$95,700/mo
```

```
COSTS
├── AI compute (with smart routing + caching):
│   ├── 60% tasks on Haiku ($0.03)            = ~$2,400
│   ├── 35% tasks on Sonnet ($0.10)           = ~$4,600
│   ├── 5% tasks on Opus ($0.17)              = ~$1,100
│   └── Prompt caching savings (-40%)         = -$3,200
│                          Compute total = ~$4,900/mo
├── Infrastructure:
│   ├── Supabase Pro                          = $25
│   ├── Vercel Pro                            = $20
│   ├── Railway                               = $100
│   ├── Redis (Upstash)                       = $50
│   ├── Pinecone                              = $70
│   └── Monitoring (Sentry + PostHog)         = $50
│                          Infra total   = ~$315/mo
├── Stripe fees (2.9% + $0.30 per transaction)   = ~$3,200/mo
└── Creator payouts (85% of rental revenue)      = ~$66,300/mo
                              TOTAL  = ~$74,715/mo

GROSS PROFIT                                  = ~$20,985/mo
GROSS MARGIN                                  = ~22%

Note: The 22% margin is AFTER paying creators their 85%.
Platform-only margin (subscriptions + overage - compute - infra) = ~93%
The marketplace cut is pure margin on top.
```

---

13. Real-Time Layer (Socket.io)

typescript

```ts
// apps/api/src/realtime/socket.ts

import { Server } from 'socket.io';

export function setupSocketIO(io: Server) {
  io.on('connection', (socket) => {
    const { orgId, userId } = socket.handshake.auth;

    // Join org room for scoped events
    socket.join(`org:${orgId}`);
    socket.join(`user:${userId}`);

    // Creator-specific room
    socket.join(`creator:${userId}`);
  });
}

// Event emitters used throughout the codebase
export const SocketEvents = {
  // Agent execution events → User dashboard
  AGENT_ACTIVITY: 'agent:activity',          // Step-by-step execution updates
  AGENT_COMPLETED: 'agent:completed',        // Execution finished
  AGENT_ERROR: 'agent:error',                // Execution failed
  APPROVAL_REQUIRED: 'agent:approval',       // Needs human approval

  // A2A events → Workflow visualizer
  A2A_CALL_STARTED: 'a2a:call_started',      // Agent calling another agent
  A2A_CALL_COMPLETED: 'a2a:call_completed',  // A2A call finished

  // Creator events → Creator dashboard
  NEW_RENTAL: 'creator:new_rental',          // Someone rented your agent
  EARNING_RECEIVED: 'creator:earning',       // Revenue from rental or A2A call
  NEW_REVIEW: 'creator:new_review',          // Someone reviewed your agent

  // Marketplace events → Browse page
  AGENT_TRENDING: 'marketplace:trending',    // Agent gaining traction
} as const;

// Example: emit A2A call for the network visualizer
export function emitA2ACall(orgId: string, data: {
  from: string;       // Caller agent ID
  to: string;         // Target agent ID
  task: string;
```

```
  status: 'started' | 'completed' | 'failed';
  result?: any;
}) {
  io.to(`org:${orgId}`).emit(SocketEvents.A2A_CALL_STARTED, data);
}

// Example: emit earning to creator
export function emitEarning(creatorId: string, data: {
  amount_cents: number;
  source: string;
  agent_name: string;
}) {
  io.to(`creator:${creatorId}`).emit(SocketEvents.EARNING_RECEIVED, data);
}
```

---

14. API Routes

Marketplace Routes

```
GET    /api/marketplace/browse         # Browse with filters & pagination
GET    /api/marketplace/search?q=...   # Text + semantic search
GET    /api/marketplace/categories     # List categories with counts
GET    /api/marketplace/featured       # Featured/trending agents
GET    /api/marketplace/agent/:slug    # Agent detail page data
```

Agent Routes (Creator)

```
POST   /api/agents                 # Create new agent
PUT    /api/agents/:id             # Update agent
POST   /api/agents/:id/publish     # Submit for publishing
DELETE /api/agents/:id             # Delete/archive agent
GET    /api/agents/mine            # List creator's agents
POST   /api/agents/:id/sandbox     # Create sandbox instance for testing
```

Rental Routes

```
POST   /api/rentals                # Rent an agent (creates Stripe subscription)
DELETE /api/rentals/:id            # Cancel rental
GET    /api/rentals/active         # List org's active rentals
PUT    /api/rentals/:id/permissions  # Update allowed connections
```

Connection Routes

```
GET    /api/connections                # List org's connections
GET    /api/connections/available      # List available connector types
GET    /api/connections/oauth/:type/start    # Start OAuth flow
GET    /api/connections/oauth/:type/callback # OAuth callback
DELETE /api/connections/:id            # Disconnect
POST   /api/connections/:id/test       # Test connection health
```

Workflow Routes

```
POST   /api/workflows/assemble         # AI Workflow Assembler
POST   /api/workflows                  # Save a workflow
GET    /api/workflows                  # List org's workflows
POST   /api/workflows/:id/execute      # Run a workflow
GET    /api/workflows/:id/status       # Get execution status
```

A2A Routes

```
GET    /api/a2a/agents/:slug/.well-known/agent.json   # A2A Agent Card discovery
POST   /api/a2a/agents/:slug/tasks                    # Create A2A task
GET    /api/a2a/agents/:slug/tasks/:taskId            # Get task status
```

Billing Routes

```
POST   /api/billing/create-checkout    # Stripe checkout for rental
POST   /api/billing/webhook            # Stripe webhook handler
GET    /api/billing/earnings           # Creator earnings summary
GET    /api/billing/earnings/history   # Earnings transaction history
POST   /api/billing/payouts/request    # Request manual payout
```

Auth Routes

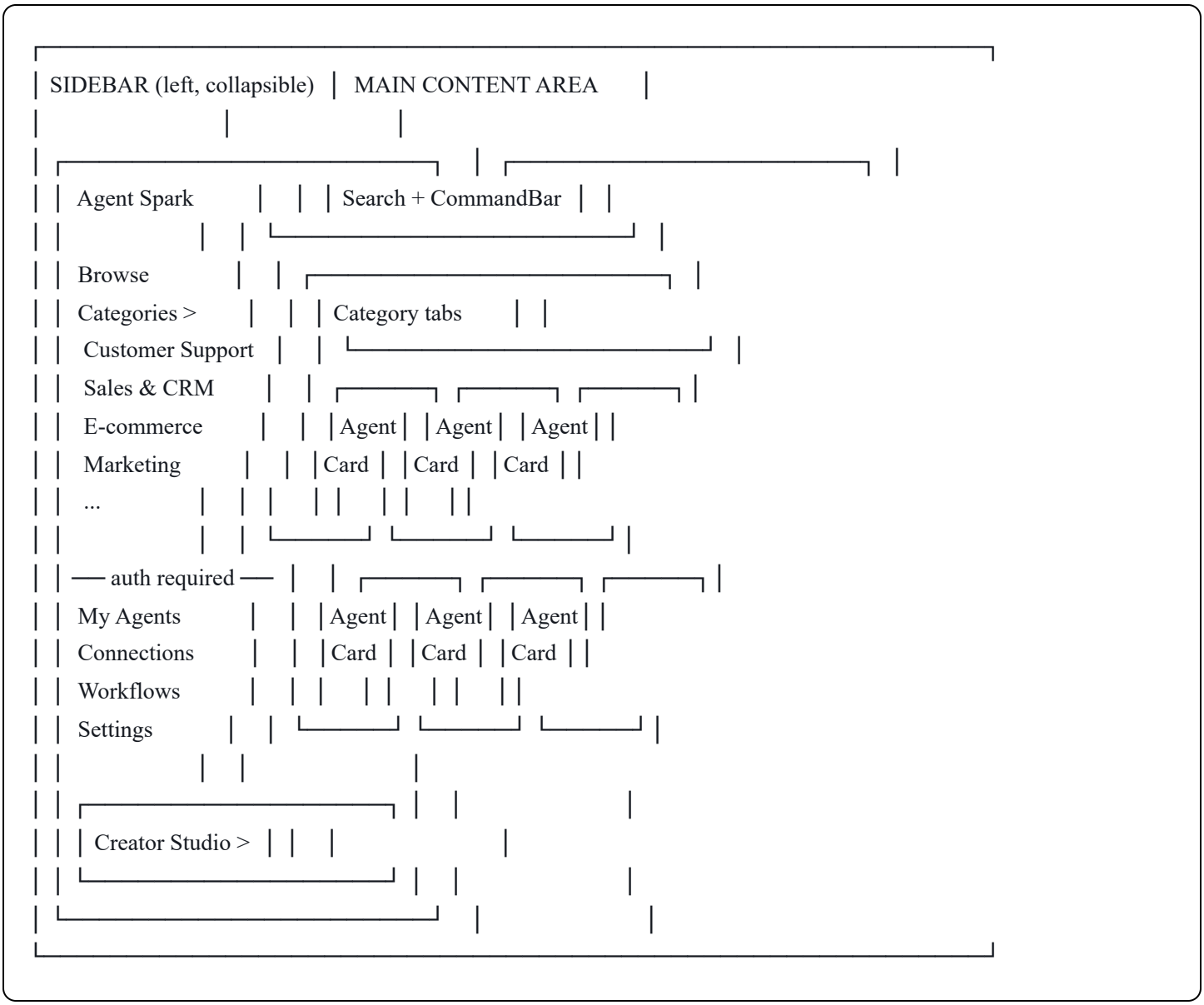
```
POST   /api/auth/signup
POST   /api/auth/login
POST   /api/auth/logout
GET    /api/auth/session
POST   /api/auth/become-creator        # Initiate Stripe Connect onboarding
```

---

15. Frontend Pages & Layout

Layout — OpenSea-Style Marketplace

The marketplace IS the homepage. No separate landing page, no marketing site. Users land directly in the product and see agents immediately. Auth wall only appears when users try to interact (rent, connect, publish).



Unauthenticated sidebar: Logo, Browse, Categories, "Log in" button **Authenticated sidebar:** Logo, Browse, Categories, My Agents, Connections, Workflows, Settings, "Creator Studio →" toggle **Creator Studio sidebar (separate mode):** "← Back to Marketplace", My Listings, Publish New, Earnings, Analytics

The CommandBar ("describe what you need") sits alongside the search bar at the top of the main content area.

Two Modes

Marketplace Mode (default): Browsing, searching, renting, using agents. Where 90% of users spend their time.

Creator Studio Mode (toggle): Publishing, managing listings, tracking earnings. Sidebar swaps to creator-specific nav. Click "← Back to Marketplace" to return.

Page Map (13 pages)

Marketplace Mode — Public (no auth)

Route	Page	Description
`/`	Marketplace Home	Agent grid, search, filters, category tabs. THE homepage.
`/category/:slug`	Category View	Filtered grid for a specific category
`/agent/:slug`	Agent Detail	Description, full radar chart, reviews, sandbox demo, rent CTA

Marketplace Mode — Authenticated

Route	Page	Description
`/dashboard`	My Agents	Rented agents, usage meter (tasks remaining), quick-execute
`/connections`	Connections	Connected tools grid, 1-click OAuth, status indicators
`/workflows`	Workflows	Active workflows, saved workflow list
`/workflows/:id`	Workflow Detail	Live agent activity feed, A2A network visualizer
`/settings`	Settings	Account, billing, plan management, upgrade flow

Creator Studio Mode — Authenticated + Creator Role

Route	Page	Description
`/creator`	Creator Home	Overview: published agents, total earnings, recent activity
`/creator/publish`	Publish Agent	3-step smart listing: paste/describe → review → publish
`/creator/listings`	My Listings	All published agents with management controls
`/creator/earnings`	Earnings	Revenue chart, transaction history, payout settings
`/creator/analytics`	Analytics	Per-agent usage stats, rating trends, A2A call volume

Key Components

```
components/
├── layout/
│   ├── Sidebar.tsx             # Collapsible sidebar with mode switching
│   ├── MarketplaceSidebar.tsx  # Browse + Categories + user nav items
│   ├── CreatorSidebar.tsx      # Creator Studio nav items
│   ├── ModeToggle.tsx          # "Creator Studio →" / "← Back to Marketplace"
│   ├── AuthWall.tsx            # Modal that appears on protected actions (not a page)
│   └── TopBar.tsx              # Search bar + CommandBar + user avatar
│
├── marketplace/
│   ├── AgentCard.tsx           # Card in browse grid (mini radar chart, price, rating)
│   ├── AgentGrid.tsx           # Responsive grid of AgentCards
│   ├── AgentRadarChart.tsx     # Recharts radar chart (5 auto-generated stats)
│   ├── CategoryNav.tsx         # Horizontal category pill tabs
│   ├── SearchBar.tsx           # Text search with autocomplete
│   ├── CommandBar.tsx          # "Describe what you need" AI input
│   ├── FilterBar.tsx           # Price, rating, connector filters (horizontal, above grid)
│   ├── AgentDetailHero.tsx     # Top section of detail page
│   ├── AgentReviews.tsx        # Review list + write review
│   ├── SandboxDemo.tsx         # Try-before-you-buy embedded agent test
│   └── RentButton.tsx          # CTA → auth check → Stripe checkout
│
├── dashboard/
│   ├── ActiveAgentsList.tsx    # Rented agents with status
│   ├── UsageMeter.tsx          # Tasks remaining this period (visual bar)
│   ├── ConnectionCard.tsx      # Connected tool with status indicator
│   ├── ConnectionGrid.tsx      # Grid + "Add New" OAuth flow trigger
│   ├── WorkflowVisualizer.tsx  # Real-time graph: agents + A2A calls
│   ├── ExecutionFeed.tsx       # Live feed of agent steps via Socket.io
│   └── ApprovalCard.tsx        # Pending approval with approve/reject
│
├── creator/
│   ├── PublishFlow.tsx         # 3-step smart listing
│   ├── ListingPreview.tsx      # Live preview of marketplace card
│   ├── ListingEditor.tsx       # Edit auto-generated listing fields
│   ├── PricingSelector.tsx     # Simple pricing picker
│   ├── EarningsChart.tsx       # Revenue over time (Recharts)
│   ├── EarningsTable.tsx       # Transaction breakdown
│   └── ListingCard.tsx         # Published agent management card
│
├── shared/
```

```
│   ├── GlassCard.tsx          # Reusable glassmorphic card
│   ├── AuthModal.tsx          # Sign up / log in modal (not a page — appears on interaction)
│   ├── UpgradeModal.tsx       # Plan upgrade prompt when hitting usage limits
│   └── LoadingStates.tsx      # Skeleton loaders for each component type
```

---

16. UI Design System

Design Tokens

Apply the glassmorphic premium design system. This is NOT a generic dashboard — it's a marketplace people want to browse.

```
css
```

```css
/* globals.css */

:root {
  /* Typography */
  --font-heading: 'Plus Jakarta Sans', sans-serif;
  --font-body: 'DM Sans', sans-serif;
  --font-mono: 'JetBrains Mono', monospace;

  /* Colors — Clean white with subtle depth */
  --bg-primary: #FFFFFF;
  --bg-secondary: #F8F9FC;
  --bg-tertiary: #F1F3F8;
  --surface-glass: rgba(255, 255, 255, 0.72);
  --surface-glass-border: rgba(255, 255, 255, 0.18);

  --text-primary: #0F1729;
  --text-secondary: #4A5568;
  --text-tertiary: #8896AB;

  --accent-primary: #6366F1;    /* Indigo */
  --accent-primary-hover: #4F46E5;
  --accent-secondary: #8B5CF6;  /* Violet */
  --accent-gradient: linear-gradient(135deg, #6366F1 0%, #8B5CF6 50%, #A78BFA 100%);

  --success: #10B981;
  --warning: #F59E0B;
  --error: #EF4444;

  /* Shadows — Multi-layer for depth */
  --shadow-sm: 0 1px 2px rgba(15, 23, 41, 0.04), 0 1px 3px rgba(15, 23, 41, 0.06);
  --shadow-md: 0 2px 4px rgba(15, 23, 41, 0.04), 0 4px 12px rgba(15, 23, 41, 0.08);
  --shadow-lg: 0 4px 8px rgba(15, 23, 41, 0.04), 0 8px 24px rgba(15, 23, 41, 0.1);
  --shadow-glow: 0 0 20px rgba(99, 102, 241, 0.15);

  /* Glass effect */
  --glass-blur: 16px;
  --glass-bg: rgba(255, 255, 255, 0.72);
  --glass-border: 1px solid rgba(255, 255, 255, 0.18);
  --glass-inner-glow: inset 0 1px 0 0 rgba(255, 255, 255, 0.5);

  /* Spacing */
  --space-card-padding: 24px;
  --space-section-gap: 48px;
```

```
  --space-page-margin: 40px;

  /* Radii */
  --radius-sm: 8px;
  --radius-md: 12px;
  --radius-lg: 16px;
  --radius-xl: 24px;
}

/* Glass Card base class */
.glass-card {
  background: var(--glass-bg);
  backdrop-filter: blur(var(--glass-blur));
  -webkit-backdrop-filter: blur(var(--glass-blur));
  border: var(--glass-border);
  border-radius: var(--radius-lg);
  box-shadow: var(--shadow-md), var(--glass-inner-glow);
  padding: var(--space-card-padding);
  transition: box-shadow 0.2s ease, transform 0.2s ease;
}

.glass-card:hover {
  box-shadow: var(--shadow-lg), var(--glass-inner-glow);
  transform: translateY(-2px);
}
```

Typography Scale

```
css

/* Headings: Plus Jakarta Sans */
h1 { font-family: var(--font-heading); font-size: 48px; font-weight: 700; letter-spacing: -0.02em; line-height: 1.1; }
h2 { font-family: var(--font-heading); font-size: 36px; font-weight: 700; letter-spacing: -0.015em; line-height: 1.2; }
h3 { font-family: var(--font-heading); font-size: 24px; font-weight: 600; letter-spacing: -0.01em; line-height: 1.3; }
h4 { font-family: var(--font-heading); font-size: 18px; font-weight: 600; letter-spacing: -0.005em; line-height: 1.4; }

/* Body: DM Sans */
body { font-family: var(--font-body); font-size: 15px; font-weight: 400; line-height: 1.6; color: var(--text-primary); }
.text-sm { font-size: 13px; }
.text-lg { font-size: 17px; }
```

Icon System

Use Lucide React exclusively. Size: 20px default. strokeWidth: 1.75. Always inside a tinted container for

context:

tsx

```tsx
// Icon container pattern
<div className="w-10 h-10 rounded-xl bg-indigo-50 flex items-center justify-center">
  <ShoppingCart className="w-5 h-5 text-indigo-600" strokeWidth={1.75} />
</div>
```

---

17. Environment Variables

bash

```
# .env.example

# Supabase
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
SUPABASE_SERVICE_ROLE_KEY=

# Anthropic
ANTHROPIC_API_KEY=

# Stripe
STRIPE_SECRET_KEY=
STRIPE_WEBHOOK_SECRET=
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=

# Redis (Upstash)
REDIS_URL=

# Pinecone
PINECONE_API_KEY=
PINECONE_INDEX=

# Voyage AI (Embeddings)
VOYAGE_API_KEY=

# OAuth — Connectors
GOOGLE_CLIENT_ID=
GOOGLE_CLIENT_SECRET=
SLACK_CLIENT_ID=
SLACK_CLIENT_SECRET=
SHOPIFY_CLIENT_ID=
SHOPIFY_CLIENT_SECRET=
HUBSPOT_CLIENT_ID=
HUBSPOT_CLIENT_SECRET=
NOTION_CLIENT_ID=
NOTION_CLIENT_SECRET=
AIRTABLE_CLIENT_ID=
AIRTABLE_CLIENT_SECRET=

# Credential Encryption
CREDENTIAL_ENCRYPTION_KEY=     # 32-byte hex key for AES-256-GCM

# App
```

```
NEXT_PUBLIC_APP_URL=http://localhost:3000
API_URL=http://localhost:4000
NODE_ENV=development

# Monitoring
SENTRY_DSN=
NEXT_PUBLIC_POSTHOG_KEY=
```

---

18. Claude Code Build Sequence (12 Steps)

Execute these in order. Each step should be completed and tested before moving to the next.

Step 1: Project Scaffolding

Set up the Turborepo monorepo with the exact structure from Section 3. Initialize Next.js 14 (App Router) in `apps/web`, Hono in `apps/api`, and the shared package in `packages/shared`. Configure TypeScript, ESLint, and Tailwind. Install all dependencies from the tech stack. Set up the design system tokens from Section 16 in `globals.css`. Import Plus Jakarta Sans and DM Sans from Google Fonts.

Step 2: Supabase Schema & Auth

Create all Supabase migrations from Section 5 — every table, every RLS policy, every index. Set up Supabase Auth with email/password and Google OAuth. Create the database trigger that auto-creates a `profiles` row and a default "Personal" `organization` on user signup. Test: user can sign up, log in, and have a profile + org created automatically.

Step 3: Frontend Shell — Sidebar Layout & Marketplace Homepage

Build the root layout with the collapsible sidebar (OpenSea-style). Implement `MarketplaceSidebar` (Browse, Categories, auth-gated items) and `CreatorSidebar` (separate mode). Build the `ModeToggle` component for switching between Marketplace and Creator Studio. Build `AuthWall` as a modal — not a page redirect — that appears when unauthenticated users try to interact (rent, connect, publish). Implement the `GlassCard` component and shared UI primitives using the design system from Section 16. Build the marketplace homepage (`/`) — this is the FIRST thing users see: `AgentGrid` with placeholder data, `CategoryNav` tabs, `SearchBar`, `FilterBar`, and `CommandBar`. The sidebar collapses on mobile. Unauthenticated users see minimal sidebar (Browse + Categories + Log in). Apply the full glassmorphic design system — this must look premium from day one.

Step 4: Connector System & OAuth

Implement the base `Connector` interface from Section 7. Build the OAuth flow handler (start + callback routes). Build the `CredentialVault` service with AES-256-GCM encryption. Implement 3 connectors to start: Gmail, Slack, Shopify. Build the Connections page in the dashboard — grid of available connectors, one-click OAuth connect flow, status indicators. Test: user can connect Gmail via OAuth and see it as "Connected."

Step 5: Smart Publish Flow & Agent Stats

Build the agent publishing backend — full CRUD for the `agents` table. Build the `PublishFlow` component — a 3-step smart listing:

Step 1 — Input: Single text area where creator either pastes their agent's system prompt OR describes what the agent does in plain English. A "Generate Listing" button sends this to Claude Sonnet which auto-generates: agent name, marketplace description, category, tags, required connectors, suggested capabilities, and recommended pricing.

Step 2 — Review & Edit: Creator sees all auto-generated fields in an editable form. Every field is pre-filled but editable. Include a `ListingPreview` component showing exactly how the agent card will appear on the marketplace (with the mini radar chart showing estimated stats).

Step 3 — Preview & Publish: Full-page preview of the agent detail page. Creator hits "Publish" and the agent is immediately live on the marketplace.

Build the `AgentRadarChart` component using Recharts `RadarChart`. This displays 5 stats scored 0-99:

Stat	Source	Calculation
Speed	`agent_executions` table	Avg execution time vs category average. Faster = higher score
Accuracy	`agent_executions` table	(completed / total executions) × 99. Success rate
Reliability	`agent_executions` table	30-day rolling success rate + low error frequency
Popularity	`rentals` table	Active rental count + 30-day retention rate, normalized to category
Versatility	`agents` table	Number of supported connectors, scaled 0-99

New agents with zero usage data get estimated scores: Speed 50 (neutral), Accuracy 50, Reliability 50, Popularity 0, Versatility calculated from connector count. Stats update on every execution via a background job.

The mini radar chart appears on every `AgentCard` in the browse grid (small, ~80px, glassmorphic background). The full radar chart with individual stat breakdowns appears on the agent detail page.

SQL function for stat calculation:

```sql
```

```sql
CREATE OR REPLACE FUNCTION calculate_agent_stats(agent_uuid UUID)
RETURNS JSONB AS $$
DECLARE
  stats JSONB;
  exec_count INTEGER;
  success_count INTEGER;
  avg_duration_ms INTEGER;
  category_avg_ms INTEGER;
  active_rentals INTEGER;
  connector_count INTEGER;
BEGIN
  -- Get execution metrics
  SELECT COUNT(*), COUNT(*) FILTER (WHERE status = 'completed'), AVG(duration_ms)::INTEGER
  INTO exec_count, success_count, avg_duration_ms
  FROM agent_executions WHERE agent_id = agent_uuid AND started_at > NOW() - INTERVAL '30 days';

  -- Category average speed
  SELECT AVG(ae.duration_ms)::INTEGER INTO category_avg_ms
  FROM agent_executions ae JOIN agents a ON ae.agent_id = a.id
  WHERE a.category = (SELECT category FROM agents WHERE id = agent_uuid)
  AND ae.started_at > NOW() - INTERVAL '30 days';

  -- Active rentals
  SELECT COUNT(*) INTO active_rentals FROM rentals WHERE agent_id = agent_uuid AND status = 'active';

  -- Connector count
  SELECT COALESCE(array_length(required_connectors, 1), 0) + COALESCE(array_length(optional_connectors, 1), 0)
  INTO connector_count FROM agents WHERE id = agent_uuid;

  stats := jsonb_build_object(
    'speed', CASE WHEN exec_count = 0 THEN 50
          WHEN category_avg_ms = 0 THEN 50
          ELSE LEAST(99, GREATEST(0, 99 - ((avg_duration_ms - category_avg_ms) * 50 / GREATEST(category_avg_ms,
    'accuracy', CASE WHEN exec_count = 0 THEN 50
            ELSE LEAST(99, (success_count * 99 / exec_count)) END,
    'reliability', CASE WHEN exec_count < 10 THEN 50
              ELSE LEAST(99, (success_count * 99 / exec_count)) END,
    'popularity', LEAST(99, active_rentals * 10),
    'versatility', LEAST(99, connector_count * 16)
  );

  -- Update the agents table
  UPDATE agents SET stats = stats, stats_updated_at = NOW() WHERE id = agent_uuid;
```

```
  RETURN stats;
END;
$$ LANGUAGE plpgsql;
```

Build the Creator's "My Listings" page showing all published agents with stats. Test: creator pastes a system prompt, AI generates a listing, creator publishes, agent appears on marketplace with estimated radar chart stats.

Step 6: Agent Runtime Engine

Port and adapt the agentic loop from Section 8. Implement `gatherConnectorTools()` to load MCP tools from the user's connected services. Set up BullMQ queues for async agent execution. Build the `ExecutionFeed` component — live feed of agent steps via Socket.io. Build the approval queue system. Test: rent a test agent, give it a task, watch it execute tools against a connected service.

Step 7: Marketplace Engine

Implement the marketplace search and browse backend from Section 10. Build the full browse page with working filters, sorting, and pagination. Build the Agent Detail page — hero, long description, reviews, required connections, pricing, "Rent This Agent" CTA. Implement agent reviews (create, read, helpful count). Set up Pinecone for semantic agent search + capability embeddings. Test: user can browse, filter, search, view agent details, and read reviews.

Step 8: Billing — Platform Subscriptions & Agent Rentals

Implement the two-stream billing model from Section 12. First, build platform subscriptions: integrate Stripe for Free/Starter/Pro/Business plans. Build the pricing page with plan comparison. Implement `checkUsageAllowance()` — this must be called before EVERY agent execution to enforce task limits. Build usage tracking (increment `tasks_used_this_period` on every execution, reset on billing cycle). Build the upgrade flow when a user hits their limit. Second, build agent rentals: implement `createRental()` with Stripe Connect for automatic 85/15 creator/platform split. Build the "Rent This Agent" checkout flow — user selects which connections to grant, pays via Stripe. Handle Stripe webhooks (subscription created, renewed, cancelled, failed). Build the User Dashboard — list of active rentals, usage meter showing tasks remaining, quick-execute buttons. Test: user can subscribe to a plan, rent an agent, get charged, see usage counting down, and hit the upgrade prompt when limit is reached.

Step 9: A2A Orchestration

Implement the `a2a-orchestrator` connector from Section 9. Register it as a tool in the agent runtime so agents can call `discover_agents` and `delegate_to_agent`. Implement A2A call metering — record every call in `a2a_calls` table, calculate earnings split. Build the `WorkflowVisualizer` component — real-time graph showing agents communicating via Socket.io. Test: Agent A can discover and delegate a task to Agent B, the call is recorded, the creator of Agent B earns revenue.

Step 10: AI Workflow Assembler

Implement the Workflow Assembler service from Section 11. Build the `CommandBar` component — global input that appears on marketplace and dashboard. When user types a description, call the assembler API, display recommended agents with explanations. Allow user to accept recommendations → auto-rent all recommended agents → create a saved workflow. Build the Workflows page in dashboard. Test: user types "automate my Shopify returns," gets recommended agents, rents them all in one flow.

Step 11: Creator Earnings, Smart Model Routing & Cost Optimization

Implement Stripe Connect onboarding for creators. Build the "Become a Creator" flow. Build the Creator Earnings dashboard — revenue chart (Recharts), transaction history table, payout status. Implement the earnings recording system — track revenue from both rentals and A2A calls. Set up the A2A payout processing job (BullMQ scheduled). Implement the Smart Model Router from Section 12 — classify task complexity with Haiku, then route to Haiku/Sonnet/Opus based on complexity. Integrate prompt caching (`cache_control: { type: 'ephemeral' }`) on all agent system prompts. Implement execution guardrails — enforce max iterations, max tokens, and max A2A calls per plan tier. Add batch processing queue for scheduled agent tasks (50% cost savings). Test: creator publishes an agent, someone rents it, creator sees earnings in real-time. Verify that simple tasks route to Haiku and complex tasks route to Sonnet/Opus.

Step 12: Polish, Monitoring & Deploy

Set up Sentry error tracking (frontend + backend). Set up PostHog analytics. Add loading skeletons for every component. Add error boundaries and fallback UIs. Responsive design pass — ensure marketplace and dashboard work on mobile. Performance optimization — image lazy loading, API response caching with Redis. Set up Vercel deployment (frontend) and Railway deployment (backend). Configure custom domain. Final design QA pass — every page must match the glassmorphic design system from Section 16.

Appendix: Seed Data

For development, seed the database with:

- 5 sample connector types (Gmail, Slack, Shopify, Notion, Google Sheets)
- 3 sample creators with published agents:
  - "Customer Support Pro" (requires: Gmail, Shopify) — $49/mo
  - "Order Tracker" (requires: Shopify) — $29/mo
  - "Email Responder" (requires: Gmail) — Free
- Sample reviews for each agent
- Sample A2A call history between agents

This enables frontend development against realistic data from day one.

This is a complete, build-ready technical PRD. Feed it to Claude Code and execute steps 1-12 in order. Each step builds on the previous one. Do not skip steps.